

Problem Statement:

Write a LEX program to read a C file. Remove all preprocessing directives and comments. Perform lexical analysis to identify keywords, identifiers, constants, strings, operators and special symbols. Tokenize each lexeme and output into a new file. Create a symbol table for the same.

Source Code:

```
%{#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_IDENT_LENGTH 100

// Define token types
typedef enum {
    KEYWORD, IDENTIFIER, CONSTANT, STRING, OPERATOR, SPECIAL,
    UNKNOWN
} TokenType;
FILE *outFile;    // Output file for tokens
FILE *symTableFile; // Output file for symbol table

// Symbol table to store identifiers and constants
typedef struct Symbol {
    char name[MAX_IDENT_LENGTH];
    TokenType type;
    struct Symbol *next;
} Symbol;

Symbol *symbolTable = NULL;
void addSymbol(const char *name, TokenType type) {
    Symbol *newSymbol = (Symbol *)malloc(sizeof(Symbol));
    strncpy(newSymbol->name, name, MAX_IDENT_LENGTH);
    newSymbol->type = type;
    newSymbol->next = symbolTable;
    symbolTable = newSymbol;
}

void writeSymbolTable() {
    Symbol *current = symbolTable;
    while (current != NULL) {
```

```

        fprintf(symTableFile, "%s\t%s\n", current->name,
            current->type == IDENTIFIER ? "IDENTIFIER" :
            current->type == CONSTANT ? "CONSTANT" : "UNKNOWN");
        current = current->next;
    }
}
}%
%%
// Regular expressions for different tokens
[ \t\n]+          ; // Skip whitespace
"/*"([^\*]|\\*+[^*/])*\*+" /" ; // Skip multiline comments
"//".*[\n]        ; // Skip single-line comments

#include|define|if|else|while|for|return|void|int|float|char { /* Keywords */ }
[a-zA-Z_][a-zA-Z0-9_]* {
    // Identifiers
    addSymbol(yytext, IDENTIFIER);
    fprintf(outFile, "IDENTIFIER: %s\n", yytext);
}
[0-9]+(\.[0-9]*)? {
    // Constants
    addSymbol(yytext, CONSTANT);
    fprintf(outFile, "CONSTANT: %s\n", yytext);
}
\"([^\\"\\]|\\.)*\" {
    // Strings
    fprintf(outFile, "STRING: %s\n", yytext);
}
[+\-*/%=&|!<>^~?;:(),{}[\]] {
    // Operators and special symbols
    fprintf(outFile, "SPECIAL: %s\n", yytext);
}
. {
    // Unknown or unrecognized characters
    fprintf(outFile, "UNKNOWN: %s\n", yytext);
}
%%

int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <input.c> <output.txt>\n", argv[0]);
        exit(1);
    }
}

```

```

}
FILE *inFile = fopen(argv[1], "r");
if (inFile == NULL) {
    perror("Error opening input file");
    exit(1);
}
outFile = fopen(argv[2], "w");
if (outFile == NULL) {
    perror("Error opening output file");
    exit(1);
}
symTableFile = fopen("symbol_table.txt", "w");
if (symTableFile == NULL) {
    perror("Error opening symbol table file");
    exit(1);
}
yyin = inFile;
yylex(); // Start scanning and processing

writeSymbolTable(); // Write symbol table to file

fclose(inFile);
fclose(outFile);
fclose(symTableFile);
return 0;
}

```

Input Code:

```

#include<stdio.h>
int main(){
int a=9;
int b=3;
printf("%d",a+b);
}

```

Output:

```
CD lab CD labb soumen
nlhcclab@nlhcclab:~/Desktop$ cd test
nlhcclab@nlhcclab:~/Desktop/test$ lex 1demo.l
nlhcclab@nlhcclab:~/Desktop/test$ gcc lex.yy.c
nlhcclab@nlhcclab:~/Desktop/test$ ./a.out
```

```
#include<stdio.h> | Include Directive
```

```
int | Keyword
```

```
main() | Function
```

```
{ | Pre-Processor
```

```
int | Keyword
```

```
a | Identifier
```

```
= | Operator
```

```
9 | Number
```

```
; | Number
```

```
int | Keyword
```

```
b | Identifier
```

```
= | Operator
```

```
3 | Number
```

```
; | Number
```

```
printf | Identifier
```

```
( | Left Parenthesis
```

```
" | Unknown
```

```
% | Unknown
```

```
d | Identifier
```

```
" | Unknown
```

```
, | Unknown
```

```
a | Identifier
```

```
+ | Operator
```

```
b | Identifier
```

```
) | Right Parenthesis
```

```
; | Number
```

```
} | Unknownnlhcclab@nlhcclab:~/Desktop/test$ ^C
```

```
nlhcclab@nlhcclab:~/Desktop/test$
```